

1. Conceptualization

Stocker

22212014 한석희 / han028@yu.ac.kr
<https://github.com/HanSatCode/Stocker/>

[Revision history]

Revision date	Version #	Description	Author
2026.03.23	1.0	Initial Document	22212014 한석희

= Contents =

1. Business purpose	
2. System context diagram	
3. Use case list	
4. Concept of operation	
5. Problem statement	
6. Glossary	
7. References	

1. Business purpose

a) Project background & Motivation

요즘 콘텐츠 시장을 보면 짧고 빠르게 즐길 수 있는 형식이 확실히 대세가 된 것 같다. ‘숏폼’ 콘텐츠처럼, 길게 집중해서 보기보다는 짧은 시간 안에 강한 재미나 보상을 얻는 쪽을 더 선호하는 분위기이다. 이러한 변화는 게임 안에서도 볼 수가 있었다. 물론 예전처럼 긴 스토리를 따라가는 게임도 여전히 의미가 있긴 하지만, 실제로 게임 플레이할 때 더 크게 영향을 주는 건 전투나 보상처럼 바로 체감할 수 있는 요소들인 것 같다고 생각한다. 짧은 순간에 승부가 갈리고, 결과가 바로 나오는 구조가 훨씬 몰입을 잘 유지시켜 준다고 느낌이였다. 또 하나 눈에 띄는 점이라면, 사람들이 심리적인 요소에 큰 흥미를 보인다는 것이다. 처음 만난 사람에게 “MBTI가 어떻게 되세요?”라고 물어보는 것이 자연스러워진 것처럼, 상대가 어떤 사람이고, 어떤 선택을 할지 예상해 보는 과정 자체를 재미로 느끼는 경우가 많은 것 같다.

이러한 흐름을 바탕으로, 본 프로젝트에서는 금융 시장을 게임의 배경으로 설정했다. 그러한 이유로는 짧은 순간의 판단이 바로 결과로 이어지고, 다른 사람들의 선택에 따라 상황이 계속 바뀐다는 점에서 이러한 요소가 잘 드러나는 환경이라고 생각했기 때문이다.

게임은 짧은 주기로 계속 변하는 시세를 중심으로 진행된다. 플레이어는 그 흐름을 보면서 매수나 매도를 빠르게 결정해야 하고, 중간에 제공되는 뉴스 등의 정보에 따라 전략적인 플레이를 할 수가 있을 것이다. 또한, 플레이 데이터를 기반으로 개인별 투자 성향을 분석하여 보여줌으로써, 단순히 게임을 하는 것을 넘어 자신의 투자 스타일을 파악하는 재미도 함께 느낄 수 있도록 할 것이다.

결과적으로, 이 게임은 친구들과 함께 가볍게 즐기거나, 다른 사람들과 경쟁하면서 실시간으로 전략을 겨루는 플레이를 목표로 한다. 짧은 시간 안에 결과가 나오는 구조를 통해 부담 없이 반복 플레이가 가능하고, 기존의 비교적 정적인 시뮬레이션 게임과는 다른 속도감 있는 재미를 제공하고자 한다.

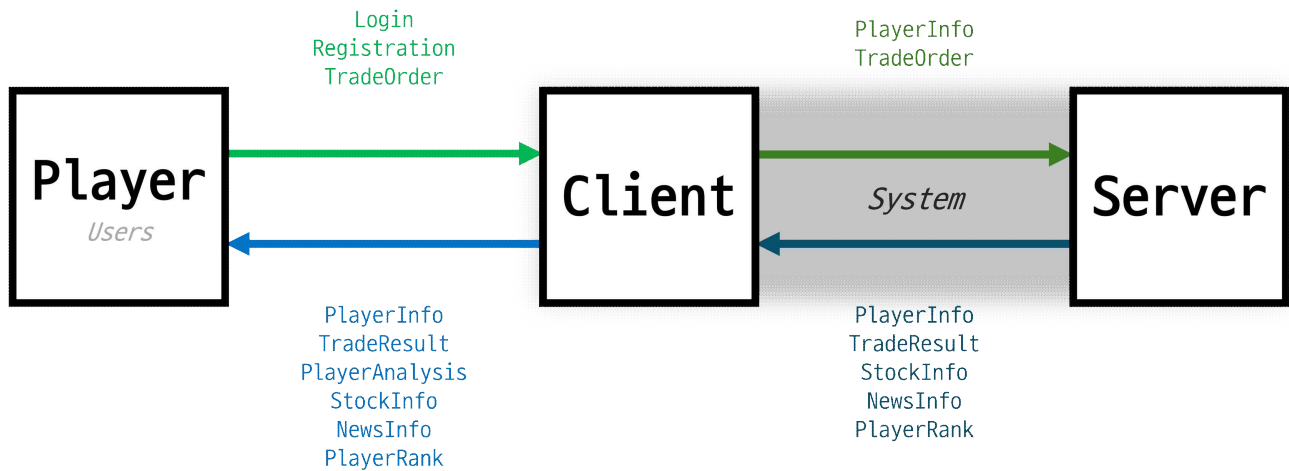
b) Goal

1. 다수의 이용자 간 상호작용을 할 수 있는 전반적인 종목 거래소 시스템을 구성한다.
2. 이용자의 매매 데이터를 분석한 투자 성향 리포트 기능을 제공한다.

c) Target market

- 빠른 호흡의 실시간 전략적 게임 플레이를 가볍게 즐기고 싶은 이용자

2. System context diagram



- Login : 플레이어 로그인
- Registration : 플레이어 (계정) 등록
- TradeOrder : 주문(매수/매도) 요청
- TradeResult : 주문(매수/매도) 요청에 따른 결과
- PlayerInfo : 플레이어 정보
- StockInfo : 종목 정보
- NewsInfo : (무작위 종목의) 뉴스 정보
- PlayerAnalysis : 플레이어의 투자 성향 데이터
- PlayerRank : 플레이어의 리더보드 데이터

3. Use case list

1) Registration

Actor	Player
Description	등록되지 않은 플레이어를 등록한다. 플레이어로부터 게임에서 사용할 닉네임 및 비밀번호를 입력받는다. 입력받은 정보는 데이터베이스에 저장된다.

2) Login

Actor	Player
Description	플레이어가 본인의 계정 정보를 입력하여 게임 시스템에 로그인한다. 서버는 입력된 정보를 바탕으로 데이터베이스에서 비교한 후, 등록된 플레이어일 경우 기존 자산 데이터와 매매 이력을 동기화하여 게임 참여 환경을 제공한다.

3) Respond Stock info

Actor	Client, Server
Description	실시간으로 서버에서 갱신되는 종목 데이터를 수신하여 클라이언트에서 시각화를 거쳐 플레이어에게 제공한다.

4) Request/Respond Buy

Actor	Player, Cilent, Server
Description	플레이어가 특정 종목에 대해 매수 버튼을 클릭하여 거래를 요청한다. 서버는 플레이어의 잔액을 확인한 후, 정상적인 거래 요청이면 플레이어의 정보를 업데이트한다. 그리고, 해당 거래에 대한 거래 기록을 수집하여 주기마다의 가격 변동에 영향을 주거나, 개인화된 투자 성향 데이터에 반영한다.

5) Request/Respond Sell

Actor	Player, Cilent, Server
Description	플레이어가 특정 종목에 대해 매도 버튼을 클릭하여 거래를 요청한다. 서버는 플레이어가 매도한 수량을 확인한 후, 정상적인 거래 요청이면 플레이어의 정보를 업데이트한다. 그리고, 해당 거래에 대한 거래 기록을 수집하여 주기마다의 가격 변동에 영향을 주거나, 개인화된 투자 성향 데이터에 반영한다.

6) Respond News data

Actor	Server, Client
Description	서버에서 무작위로 생성된 뉴스(시장 정보)를 클라이언트의 뉴스 항목에 표시한다.

7) Respond Player analysis report

Actor	Server, Client
Description	누적된 플레이어의 매매 데이터를 분석하여, 시각적인 형태로 가공한 투자 리포트(포트폴리오)를 플레이어에게 제공한다.

8) Respond Player rank

Actor	Server, Client
Description	게임에 참여 중인 모든 플레이어 정보를 분석하여, 총 자산이나 수익률 등의 순위를 시각적인 형태로 제공한다.

4. Concept of operation

1) 플레이어 등록

Purpose	신규 플레이어가 게임 시스템에 접근하기 위한 고유 계정을 생성하고 초기 자본을 할당받기 위함이다.
Approach	플레이어는 희망하는 닉네임과 비밀번호를 입력한다. 닉네임은 중복을 허용하지 않는다. 정상적인 정보를 받은 경우, 서버 내 데이터베이스에 저장된다.
Dynamics	최초로 게임 서비스에 접근하여 계정이 존재하지 않는 플레이어가 신규 가입을 시도하는 경우이다.
Goals	회원가입 기능을 통해 모든 플레이어가 서로 다른 계정으로 로그인할 수 있도록 한다.

2) 플레이어 로그인

Purpose	게임에 참여하기 위해, 각 플레이어를 식별하고 기존 자산 데이터를 불러오기 위함이다.
Approach	플레이어로부터 닉네임과 비밀번호를 입력받고, 입력된 정보를 바탕으로 서버의 데이터베이스를 검색하여 플레이어의 등록 여부를 확인한다. 유효한 플레이어 정보가 있는 경우, 플레이어는 클라이언트를 통해 해당 계정의 정보를 볼 수 있다. 등록이 확인되지 않으면, 오류 메시지를 출력한다.
Dynamics	등록된 계정을 통해 게임에 접속을 시도하는 경우이다.
Goals	각 플레이어가 로그인하여 해당 계정의 정보를 정상적으로 받아올 수 있게 한다.

3) 실시간 시세 동기화

Purpose	모든 플레이어가 서버의 종목 데이터베이스의 정보를 실시간으로 공유받기 위함이다.
Approach	서버의 시뮬레이션 엔진이 특정 시간 단위로 갱신된 종목별 현재가를 WebSocket 프로토콜을 사용하여 클라이언트에 실시간으로 전송한다.
Dynamics	서버 내부의 가격 결정 알고리즘에 의해 데이터 변동 이벤트가 발생하거나, 특정 플레이어의 대량 거래로 인한 가격 충격이 발생했을 때 구동된다.
Goals	모든 플레이어가 실시간 시세 데이터를 공유받을 수 있게 한다.

4) 플레이어 정보 동기화

Purpose	플레이어의 최신 상태 및 투자 성향 데이터를 클라이언트와 일치시켜 정확한 게임 환경을 제공하기 위함이다.
Approach	매수/매도 주문이 체결되거나 랭킹이 변동될 때, 서버는 데이터베이스에서 갱신된 정보를 추출하여 클라이언트에 전송하며, 클라이언트에서는 해당 정보를 시각화하여 플레이어에게 제공한다.
Dynamics	거래 체결 직후, 혹은 플레이어가 자신의 프로필을 확인하는 등 최신 정보를 요청할 때 발생한다.
Goals	플레이어에게 자신의 실시간 자산 현황을 시각화하여 제공한다.

5) 매수 주문

Purpose	플레이어가 자산을 주식 지분으로 전환하기 위함이다.
Approach	플레이어가 종목과 수량을 선택해 매수 요청을 보내면, 서버는 플레이어의 잔액 확인 후 자산을 차감하고 해당 거래를 승인한다.
Dynamics	플레이어가 매수 버튼을 누르는 경우이다.
Goals	플레이어에게 정확한 수량을 할당한다.

6) 매도 주문

Purpose	보유 주식을 매도하여 수익을 실현하기 위함이다.
Approach	플레이어가 종목과 수량을 선택해 매도 요청을 보내면, 서버는 플레이어의 보유 종목 및 수량을 확인한 후 실시간 시세에 맞추어 자산을 가산한다.
Dynamics	플레이어가 매도 버튼을 누르는 경우이다.
Goals	플레이어에게 정확한 차익을 반환한다.

7) 뉴스 수신

Purpose	정보의 불확실성을 제공하여 플레이어 간의 심리적 싸움을 유도하기 위함이다.
Approach	서버에서 무작위로 생성한 정보를 모든 클라이언트에 전송한다.
Dynamics	특정 알고리즘에 의해 시장에 영향을 주는 변수가 발생한 경우이다.
Goals	시장의 반응을 끌어내는 뉴스 기능을 구현한다.

8) 투자 성향 분석

Purpose	플레이어의 거래 데이터를 기반으로 본인의 투자 성향을 보여주기 위함
Approach	누적된 매매 패턴을 분석하여 성향 데이터를 생성한다.
Dynamics	플레이어가 본인의 분석 데이터 확인을 요청하거나, 충분한 거래 데이터가 확보된 경우이다.
Goals	각 플레이어에게 고유한 데이터를 제공한다.

9) 플레이어 자산 정보 확인 (포트폴리오)

Purpose	플레이어가 본인의 자산 정보를 보기 위함
Approach	클라이언트는 서버로부터 해당 플레이어의 정보를 가져온 후, 수치 및 시각적인 형태로 제공한다.
Dynamics	플레이어가 본인의 자산 확인 요청을 요청하는 경우이다.
Goals	각 플레이어 본인의 자산 데이터를 제공한다.

5. Problem statement

Overview)

‘Stocker’는 웹 플랫폼을 기반으로 한 시뮬레이션 게임 서비스이며, 모든 데이터는 실시간으로 처리 및 동기화된다. 해당 서비스를 구성하는 과정에서 여러 가지 문제가 발생할 것으로 예상된다.

Problem 01 - 서버 성능 문제 및 과부하

짧은 시간 단위로 변하는 주가 데이터를 모든 클라이언트에 실시간으로 동시에 보내야 하는 게임 서비스 특성상, 플레이어 수가 증가할수록 서버 자원 소모가 극심해지는 문제가 발생한다. 특히, 사용자 정보나 랭킹 시스템 등의 데이터까지 실시간으로 처리할 경우, 서버의 연산과 네트워크 부하는 더더욱 가중될 것이다.

Problem 02 - 보안 및 데이터 변조

본 프로젝트는 실시간 게임 서비스 특성상 데이터의 신뢰성이 매우 중요한데, 비밀번호 유출 위험과 클라이언트 측의 악의적인 데이터 변조로 인한 게임 서비스 붕괴 등에 대해 방지책을 마련해야 한다.

NFRs)

1. **플랫폼 환경** - 웹 브라우저를 플랫폼으로 사용하며, 서버는 FastAPI를 활용한다.
2. **동기화 성능** - 서버와 클라이언트 간의 데이터 동기화 시간은 플레이어 체감상 즉각적이어야 한다.
3. **호환성 확보** - 다양한 브라우저 환경에서도 게임 화면이 깨지지 않고 정상적으로 진행할 수 있어야 한다.

6. Glossary

용어	설명
숏폼(Short-form)	짧은 길이의 콘텐츠. 주로 영상 콘텐츠에 해당함.
시뮬레이션 엔진	서버 내 일정한 시간 주기에 따라 종목의 가격 변동 및 뉴스 생성 로직 등을 처리하는 게임의 핵심 부분.
가격 충격	플레이어의 대량 주문이 시장의 균형을 깨뜨려 즉각적인 시세 변동을 일으키는 현상.
투자 성향 리포트	플레이어의 주문 기록 등의 누적 된 데이터를 분석하여 시각화한 개인 맞춤형 분석 결과물.
리더보드	전체 플레이어의 자산 및 수익률을 정기적으로 집계하여 순위로 보여주는 시스템.
포트폴리오	플레이어가 보유한 자산의 상태.
FastAPI	Python 기반의 웹 프레임워크.
WebSocket	클라이언트와 서버 간에 연결을 유지한 상태로 데이터를 양방향 실시간 전송을 하기위한 프로토콜.

7. References

1. React references - <https://ko.react.dev/reference/react>
2. FastAPI references - <https://fastapi.tiangolo.com/ko/reference/>
3. Python asyncio - <https://docs.python.org/3/library/asyncio.html>